Overview of Statistical Learning and Modeling – 36-600

Lab 5R: Linear Regression
Due Friday, September 26, 11:59pm

Trishla Nair

To answer the questions below, it will help you to refer to the class notes and to Chapter 3 of ISLR (especially Section 6).

Data

We'll begin by importing the heart-disease dataset, removing the `id` column, and log-transforming the response variable, `Cost`. We'll also form `Drugs` and `Complications` factor variables.

```
df <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/heart_diseas
               stringsAsFactors = TRUE)
df <- df[, -10]
w  <- which(df$Cost > 0)
df <- df[w, ]
df$Cost          <- log(df$Cost)
df$Complications <- factor(df$Complications)
df$Drugs         <- factor(df$Drugs)
summary(df)
```

```
##       Cost             Age           Gender    Interventions    Drugs
##  Min.   : 0.470   Min.   :24.00   Female:608   Min.   : 0.000   0:608
##  1st Qu.: 5.090   1st Qu.:55.00   Male  :177   1st Qu.: 1.000   1: 88
##  Median : 6.235   Median :60.00                Median : 3.000   2: 89
##  Mean   : 6.314   Mean   :58.73                Mean   : 4.722
##  3rd Qu.: 7.582   3rd Qu.:64.00                3rd Qu.: 6.000
##  Max.   :10.872   Max.   :70.00                Max.   :47.000
##     ERVisit       Complications Comorbidities       Duration
##  Min.   : 0.000   0:742         Min.   : 0.000   Min.   :  0.0
##  1st Qu.: 2.000   1: 43         1st Qu.: 0.000   1st Qu.: 42.0
##  Median : 3.000                 Median : 1.000   Median :167.0
##  Mean   : 3.424                 Mean   : 3.781   Mean   :164.7
##  3rd Qu.: 5.000                 3rd Qu.: 5.000   3rd Qu.:281.0
##  Max.   :20.000                 Max.   :60.000   Max.   :372.0
```

Question 1

Split the data into training and test sets. Call these `df.train` and `df.test`. Assume that 70% of the data will be used to train the linear regression model. Recall that

```
s <- sample(nrow(df), round(0.7 * nrow(df)))
```

will randomly select the rows for training. Also recall that

`df[s, ] and df[-s, ]`

are ways of filtering the data frame into the training set and the test set, respectively. (Remember to set the random number seed!)

```
set.seed(42)
s <- sample(nrow(df), round(0.7 * nrow(df)))
df.train <- df[s, ]
df.test  <- df[-s, ]
```

Before moving on to performing linear regression, we should talk a bit about model syntax. Here, we'll show this syntax within the context of a simple analysis:

```
> lm.out <- lm(Cost ~ ., data = df.train)
> summary(lm.out)
> Cost.pred <- predict(lm.out, newdata = df.test)
```

Let's break this down.

First, we call `lm()`, which stands for "linear model." For our model, we decide to regress the variable `Cost` onto all the predictor variables (represented by the "."). (*Note*: that's a tilde before the period, not a minus sign! See below for what we would do if we don't want to include all the predictor variables when learning the model.) `R` doesn't know where these predictor variable are, so we specify that via the `data =` argument. We save the output as `lm.out`.

Second, we call the `summary()` function. `summary()` is a general function whose behavior depends on the class of object passed to it. If the object is a data frame, you get a numerical summary. (Try it with `df.train`!) If the object is of class `lm`, then you get entirely different output. (Basically, you can think of it as `summary()` checking for the class, then calling another function depending on what the class is. Here, `summary()` invisibly redirects `lm.out` to another function called `summary.lm()` — conveniently, you didn't even need to know that `summary.lm()` existed, `R` just calls the right tool for summarization.) The `summary()` function provides the p-values for the individual coefficients and for the F statistic, plus the adjusted R^2, etc.

Third, we use the model embedded within `lm.out` to generate predictions for the Cost on new data (the test data). `predict()` behaves like `summary()`; here, it redirects the arguments to a hidden `predict.lm()` function, appropriate for a model of this type.

Now, about the model syntax. For simplicity, assume that we have a data frame `p`, with columns `a`, `b`, and `c`, and `r` (the response variable).

- To include `a` as the only predictor: `lm(r ~ a, data = p)`
- To include `a` and `c` only: `lm(r ~ a + c, data = p)` or `lm(r ~ . - b, data = p)`
- To indicate that all variables other than `r` are predictor variables: `lm(r ~ ., data = p)`
- To regress through the origin: `lm(r ~ . - 1, data = p)`
- To include `a`, `b`, and their interaction: `lm(r ~ a + b + a:b, data = p)`

Question 2

Perform a multiple linear regression analysis. Use the `summary()` function to examine the output. Do you conclude that the linear model is informative or uninformative?

```
lm.out <- lm(Cost ~ ., data = df.train)
summary(lm.out)
```

```
##
## Call:
## lm(formula = Cost ~ ., data = df.train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.2969 -0.7028  0.0254  0.6197  3.2655
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)     5.101294   0.431156  11.832  < 2e-16 ***
## Age            -0.010451   0.007309  -1.430 0.153323
## GenderMale     -0.110811   0.119579  -0.927 0.354510
## Interventions   0.184280   0.009376  19.655  < 2e-16 ***
## Drugs1          0.142016   0.161896   0.877 0.380767
## Drugs2         -0.193004   0.185259  -1.042 0.297967
## ERVisit         0.079426   0.022612   3.513 0.000481 ***
## Complications1  0.870066   0.206152   4.221 2.86e-05 ***
## Comorbidities   0.045768   0.010056   4.551 6.59e-06 ***
## Duration        0.002920   0.000482   6.059 2.58e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.16 on 540 degrees of freedom
## Multiple R-squared:  0.6166, Adjusted R-squared:  0.6102
## F-statistic: 96.49 on 9 and 540 DF,  p-value: < 2.2e-16
```

```
Cost.pred <- predict(lm.out, newdata = df.test)
```

```
This model is informative.
```

Question 3

Interpret the linear regression model. Specifically: which three variables appear to be, in *relative terms*, the three that have the most "predictive ability"? (Do *not* include the `(Intercept)` here — we usually just ignore it when it comes to explaining relative "predictive power"!)

```
The 3 variables are drugs, interventions, and age.
```

Question 4

Does the assumption that the data are normally distributed around the regression line hold here? (To be clear: linear modeling is fine if the assumption of normality does not hold, so long as the average value of the

error term ϵ is zero and the variance of $Y|x$ is σ^2. All that non-normality means is that you cannot "trust" the hypothesis tests in the summary output in, e.g., the third and fourth columns of the coefficients table.) Make a histogram of your fitted residuals, $Y_i - \widehat{Y}_i$, for the *test-set data only*.

The following code may help you. To generate predictions for the test-set response values, take your `lm()` output and use it as follows, e.g.:

```
Cost.pred <- predict(lm.out, newdata = df.test)
```

The data frame for plotting is

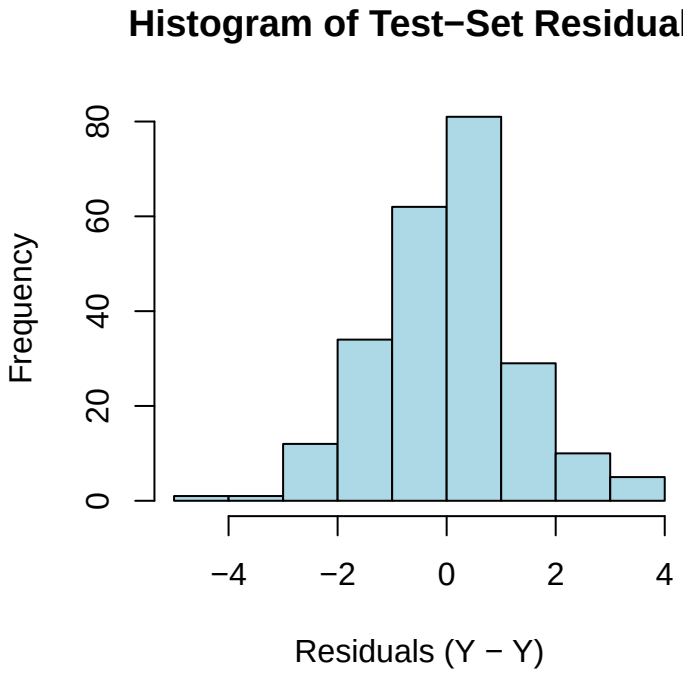
```
df.plot <- data.frame("x" = df.test$Cost - Cost.pred)
```

Pass that data frame into `ggplot()` and make a histogram.

```
suppressMessages(library(tidyverse))
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
residuals_test <- df.test$Cost - Cost.pred
hist(residuals_test,
     main = "Histogram of Test-Set Residuals",
     xlab = "Residuals (Y - Ŷ)",
     col = "lightblue",
     border = "black")
```



Because most of the residuals lie around 0, we can say that the regression line holds and that the data

Question 5

Use a Shapiro-Wilk test to decide, using numeric evidence, whether the residuals are normally distributed. What do you conclude? (Note: you will pass `df.plot$x` into the test.)

```
shapiro.test(residuals_test <- df.test$Cost - Cost.pred)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals_test <- df.test$Cost - Cost.pred
## W = 0.98744, p-value = 0.03728
```

```
Based on the Shapiro-Wilk test, because the p-value is extremely small, we can conclude that the data is
```

Question 6

Is the assumption of constant variance (assumption 2 in the notes) met with these data? Plot the model residuals versus the predicted model values, for the test set. The data frame for plotting is

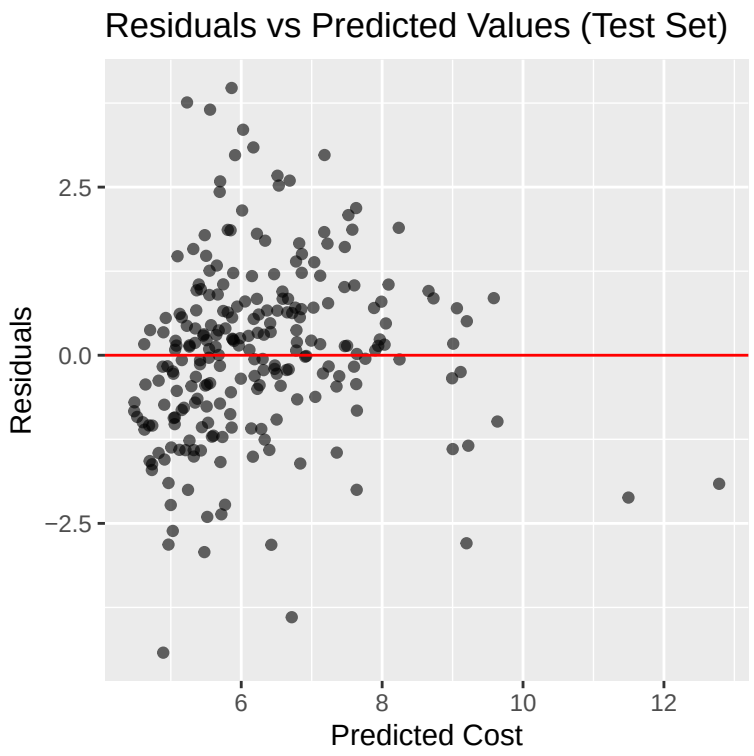
```
df.plot <- data.frame("x" = Cost.pred, "y" = df.test$Cost - Cost.pred)
```

Pass this into `ggplot()` and make a scatterplot of y vs. x.

```
library(ggplot2)

df.plot <- data.frame("x" = Cost.pred,"y" = df.test$Cost -Cost.pred)

ggplot(df.plot, aes(x = x, y = y)) +
  geom_point(alpha = 0.6) +
  geom_hline(yintercept = 0, color = "red") +
  labs(
    title = "Residuals vs Predicted Values (Test Set)",
    x = "Predicted Cost",
    y = "Residuals")
```



Question 7

While we did not explicitly mention this in the class notes, there are, as you might expect, off-the-shelf statistical tests available for assessing whether or not our data exhibit non-constant variance. One such test is the so-called *Breusch-Pagan test*. The null hypothesis for this test is the variance is constant as a function of the predicted response values.

To run this test in `R`: load the `car` library (after installing it, if necessary) and pass the linear regression output that you generated in Question 2 to the function `ncvTest()`. Interpret your result.

```
suppressMessages(library(car))
```

```
## Warning: package 'car' was built under R version 4.4.3
```

```
## Warning: package 'carData' was built under R version 4.4.3
```

```
ncvTest(lm.out)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 11.99262, Df = 1, p = 0.00053412
```

```
Since the p-value is so large, we fail to reject the null hypothesis and that the data is homosecedastic
```

Question 8

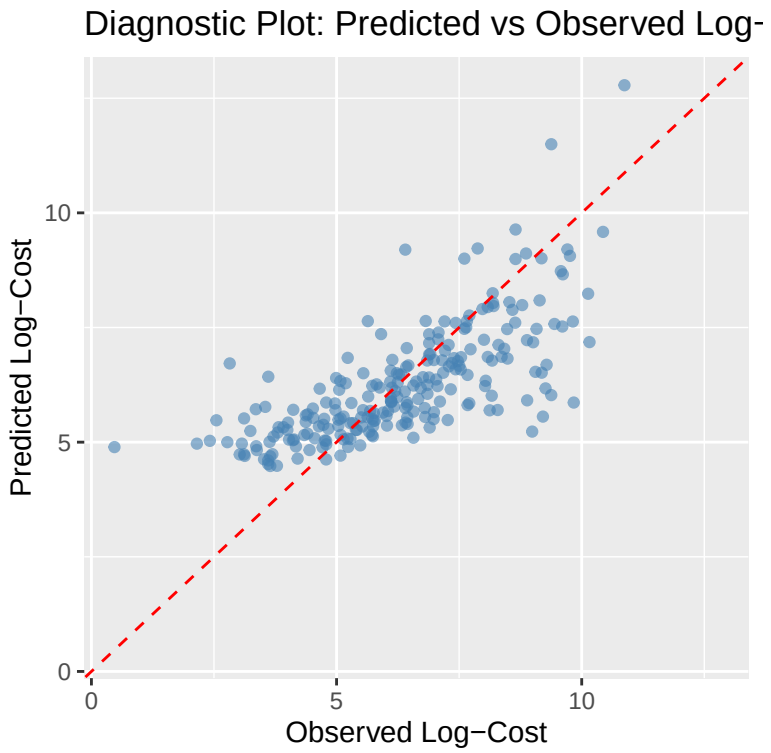
Create a diagnostic plot showing the predicted log-cost (y-axis) versus the observed log-cost (x-axis). As in the example in the notes, make the limits the same along both axes (use `xlim()` and `ylim()`) and draw a diagonal line from lower-left to upper-right (look up `geom_abline()`). Where does the linear model tend to break down the most?

The following code may help you. The data frame for plotting is

```
df.plot <- data.frame("x" = df.test$Cost, "y" = Cost.pred)
```

```
df.plot <- data.frame("x" = df.test$Cost, "y" = Cost.pred)

# Diagnostic plot: observed (x) vs predicted (y)
ggplot(df.plot, aes(x = x, y = y)) +
  geom_point(alpha = 0.6, color = "steelblue") +    # scatterplot points
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") + # 45° line
  xlim(min(df.plot$x, df.plot$y), max(df.plot$x, df.plot$y)) +
  ylim(min(df.plot$x, df.plot$y), max(df.plot$x, df.plot$y)) +
  labs(
    x = "Observed Log-Cost",
    y = "Predicted Log-Cost",
    title = "Diagnostic Plot: Predicted vs Observed Log-Cost"
  )
```



On the lower end, the model tends to over predict while on the opposite end, the moddel under predict.

Question 9

Use the `vif()` function of the `car` package (already loaded!) to check for possible multicollinearity. (Again, you are passing in your linear regression output.) Does there appear to be any issue with multicollinearity in these data? There is no need to mitigate it if you see it.

```
vif(lm.out)
```

```
##                    GVIF Df GVIF^(1/(2*Df))
## Age            1.027415  1        1.013615
## Gender         1.026023  1        1.012928
## Interventions 1.235524  1        1.111541
## Drugs          1.353845  2        1.078679
## ERVisit        1.499544  1        1.224559
## Complications 1.062161  1        1.030612
## Comorbidities 1.372498  1        1.171537
## Duration       1.415497  1        1.189747
```

```
All the vif values are pretty low, so there is no need to worry about multicolinearity.
```

Question 10

Compute the mean-squared error for the linear model. (Remember: MSEs are computed on the test-set data only.) Note that the square root of this quantity is the average distance between the predicted log-cost and the observed log-cost. We'll likely come back to these data later and see if a machine-learning model does a better job than linear regression in predicting log-cost.

```
mse <- mean(residuals_test^2)
rmse <- sqrt(mse)
print(mse)
```

```
## [1] 1.717988
```

```
print(rmse)
```

```
## [1] 1.310721
```